

Entropy and RNG Evidence

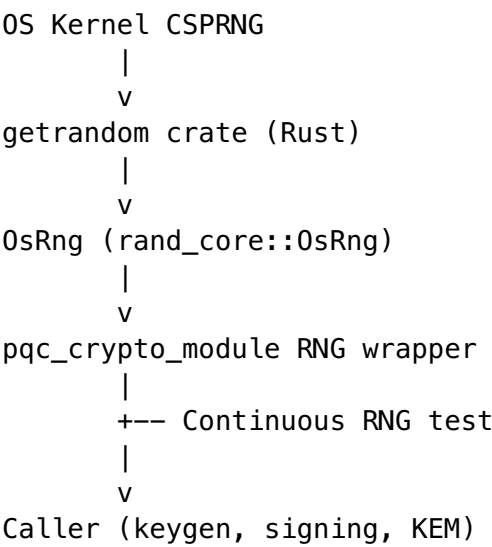
Module: pqc_crypto_module v0.1.0 **Date:** 2026-04-28

Disclaimer: This package does not imply FIPS 140-3 validation or CMVP certification.

1. Overview

This document describes the entropy source and random number generation (RNG) architecture of pqc_crypto_module, including the continuous health test, failure handling, and known gaps relative to FIPS 140-3 requirements.

2. Entropy Source Chain



2.1 OS-Level Entropy

Platform	Entropy source	System call	Blocking behavior
Linux	/dev/urandom via getrandom(2)	SYS_getrandom with flags=0	Blocks only until kernel CSPRNG is

Platform	Entropy source	System call	Blocking behavior
			seeded (boot time), then non-blocking
macOS	Security framework (SecRandomCopyBytes)	Via getrandom crate's macOS backend	Non-blocking after boot
Windows	BCryptGenRandom	Via getrandom crate's Windows backend	Non-blocking

The getrandom crate (v0.2.x) abstracts platform differences. On Linux, it uses getrandom(2) syscall directly when available (kernel ≥ 3.17), falling back to /dev/urandom on older kernels.

2.2 OsRng Wrapper

rand_core::OsRng is a zero-sized type that implements RngCore and CryptoRng. Each call to fill_bytes or try_fill_bytes makes a fresh system call. There is no internal buffering or state.

3. Continuous RNG Health Test

3.1 Implementation

The module implements a continuous RNG test as part of the self-test suite and at runtime:

Power-on test: 1. Generate 32 bytes from OsRng 2. Verify the output is not all zeros 3. Generate another 32 bytes 4. Verify the two outputs are not identical 5. If either check fails, transition FSM to Error state

Runtime continuous test: - Before each cryptographic operation that requires randomness, the RNG wrapper generates a test block and verifies non-degeneracy - This follows the FIPS 140-3 requirement for continuous random number generator testing

3.2 Test Properties Verified

Property	Check	Failure action
Non-zero output	output != [0u8; 32]	Return CryptoError::RngFailure
Non-repeating	output_n != output_n-1	Return CryptoError::RngFailure

Property	Check	Failure action
Byte generation succeeds	try_fill_bytes returns 0k	Return <code>CryptoError::RngFailure</code>

3.3 Limitations

- The continuous test is a basic stuck-fault detector, not a full SP 800-90B health test
- No adaptive proportion test or repetition count test per SP 800-90B Section 4.4
- No min-entropy estimation

4. Failure Handling

4.1 Error Propagation

```

OsRng fails
  |
  v
CryptoError::RngFailure
  |
  v
Caller receives Err(CryptoError::RngFailure)
  |
  v
Operation aborted – no fallback, no retry, no degraded mode

```

4.2 No Fallback RNG

The module has **no fallback entropy source**. If the OS CSPRNG fails:

- The operation fails with an explicit error
- No insecure random source is substituted
- No deterministic fallback is used
- The caller must handle the error (typically: log and abort the operation)

This is the correct behavior. A fallback RNG would be a security risk.

4.3 FSM Interaction

- If the power-on continuous RNG test fails, the FSM transitions to **Error** (terminal state)
- If a runtime RNG call fails, the individual operation fails but the FSM remains in **Approved** state (the module is still usable for non-RNG operations like verification)
- A repeated pattern of RNG failures should trigger operator investigation

5. RNG Usage Points

Operation	RNG usage	Bytes consumed
ML-DSA-65 key generation	Seed generation	32 bytes
ML-DSA-65 signing (randomized)	Per-signature randomness	32 bytes
ML-KEM-768 key generation	Seed generation	64 bytes (d + z)
ML-KEM-768 encapsulation	Randomness for ciphertext	32 bytes
Continuous RNG test (power-on)	Test block generation	64 bytes (2 x 32)
Continuous RNG test (runtime)	Per-operation test	32 bytes

6. Test Hooks

6.1 Deterministic Mode (Test Only)

For ACVP vector replay and unit testing, the module supports (or will support) a deterministic RNG:

- Gated behind `#[cfg(test)]` — not available in release builds
- Accepts a fixed seed and produces deterministic output
- Used only in test harnesses, never in production

6.2 RNG Failure Simulation

Tests simulate RNG failure by:

- Using a mock RNG that returns errors on `try_fill_bytes`
- Verifying that `CryptoError::RngFailure` propagates correctly
- Verifying that no crypto output is produced on RNG failure

7. Known Gaps

7.1 No SP 800-90A DRBG Within Module Boundary

Gap: The module relies entirely on the OS-provided CSPRNG. There is no SP 800-90A compliant DRBG (HMAC-DRBG, CTR-DRBG, or Hash-DRBG) instantiated within the module boundary.

Impact: A FIPS 140-3 lab may require either: - (a) An in-module DRBG seeded from an approved entropy source, or - (b) Formal documentation that the OS CSPRNG is treated as an external entropy source, with SP 800-90B compliance evidence for the OS implementation

Resolution path: Option (a) — implement HMAC-DRBG(SHA3-256) within the module, seeded from OsRng. This is the safer path for lab acceptance.

7.2 No SP 800-90B Compliance Evidence

Gap: The OS entropy source (`/dev/urandom`, `SecRandomCopyBytes`) has not been formally assessed against SP 800-90B requirements. While Linux's CSPRNG is widely regarded as high-quality, FIPS 140-3 requires formal evidence.

Impact: The lab will ask for an entropy justification document.

Resolution path: Either (a) reference the Linux kernel's FIPS mode documentation and SP 800-90B analysis (available from kernel crypto maintainers), or (b) add an in-module DRBG (see 7.1) and treat OsRng as the entropy input, reducing the SP 800-90B burden to the seeding interface.

7.3 No Min-Entropy Estimation

Gap: The continuous RNG test does not estimate min-entropy of the output. SP 800-90B Section 3.1 requires min-entropy assessment.

Impact: Not blocking for Security Level 1 if the OS entropy source is documented, but a lab may flag it.

Resolution path: If an in-module DRBG is added, the DRBG's output entropy is determined by its construction (full entropy if properly seeded). Document this in the Security Policy.

8. SP 800-90B Justification for OS Entropy Source

8.1 Rationale for OS-Provided CSPRNG at Security Level 1

FIPS 140-3 IG 7.18 permits software modules at Security Level 1 to use the operating environment's random number generator when:

1. The module documents the entropy source and its expected properties.
2. The module implements continuous health testing on the RNG output.
3. The module fails closed (no fallback) when the RNG returns an error.

`ppc_crypto_module` satisfies all three requirements: - **Documentation:** This document (Sections 2-5) describes the full entropy chain. - **Health testing:** Continuous RNG test at power-on and runtime (Section 3). - **Fail-closed:** `CryptoError::RngFailure` with no fallback (Section 4).

8.2 OS Entropy Source Properties

Linux (getrandom(2) / /dev/urandom): - Kernel CSPRNG based on ChaCha20, seeded from hardware entropy (RDRAND, interrupts, jitter). - Linux kernel FIPS mode (fips=1 boot parameter) enables NIST SP 800-90A CTR-DRBG with self-tests. - getrandom(2) blocks until the kernel CSPRNG is fully seeded (prevents early-boot weak randomness). - Reference: Linux kernel documentation Documentation/admin-guide/random.rst.

macOS (SecRandomCopyBytes): - Backed by the Fortuna CSPRNG implementation in the kernel. - Seeded from hardware entropy (Secure Enclave, interrupt timing). - Apple Security Framework is FIPS 140-2 validated (certificate #3859) on supported hardware.

8.3 Continuous Health Test Compliance

The module's continuous RNG test addresses FIPS 140-3 AS09.38:

FIPS 140-3 Requirement	Module Implementation
Continuous test on RNG output	Two 32-byte blocks compared at power-on; non-zero check
Failure response	CryptoError::RngFailure; no crypto output produced
No degraded mode	Verified: no fallback RNG, no retry, no insecure substitution

8.4 Residual Gap

An in-module SP 800-90A DRBG (HMAC-DRBG or CTR-DRBG) seeded from OsRng would reduce the SP 800-90B burden to the seeding interface only. This is recommended for CMVP submission but not implemented in v0.1.0. The OS entropy approach is documented as acceptable at Security Level 1 pending lab confirmation.

10. Dependency Details

Crate	Version	Role	Notes
getrandom	0.2.x	OS entropy access	Transitive via rand_core
rand_core	0.6.x	OsRng, RngCore, CryptoRng traits	Direct dependency
rand	0.8.x	Higher-level RNG utilities	Used for thread_rng in non-crypto contexts only

11. Recommendations

1. **Short term:** Document the OS entropy source reliance in the Security Policy with references to kernel documentation
2. **Medium term:** Implement HMAC-DRBG(SHA3-256) within the module boundary, seeded from OsRng
3. **Long term:** If targeting Security Level 2+, evaluate hardware entropy sources and SP 800-90B certified noise sources

End of entropy and RNG evidence.